From Encrypted Flows to Security Event Signals: An Edge-Native Pipeline for Threat-Aware Traffic Analysis

Yu-Lun Tsai[1], Yu-Liang Cheng[1], Yu-Ying Huang[1], and Ching-Hao Mao[1]

[1]Bachelor's Program in Artificial Intelligence and Information Security, Fu Jen Catholic University, New Taipei City, Taiwan

[1]412580084@cloud.fju.edu.tw (Y.-L. Tsai); 412580292@cloud.fju.edu.tw (Y.-L. Cheng); 412580060@cloud.fju.edu.tw (Y.-Y. Huang); chmao2008@gmail.com (C.-H. Mao)

摘要

Widespread HTTPS/TLS limits cleartext visibility at enterprise and carrier edges, so operators must infer risk from flow-level metadata, timing, and side-channel structure rather than payloads. Encrypted traffic classifiers frequently report a single label or score vector, yet security operations increasingly require *structured security event signals*: versioned records that bind identity, network context, calibrated model output, threat semantics, and provenance for correlation, ticketing, and later reasoning.

This paper is *not* a study of a specific vendor telemetry product nor a contest among neural encoders; it examines how encrypted flow-level predictions can be *transformed* into threshold-consistent, schema-stable events suitable for edge-native defense stacks. We instantiate a two-stage pipeline: Stage 1 benign/malicious screening with calibrated dispatch, then Stage 2 threat-behavior labeling on the admitted subset, each flow emitting a normalized record spanning identity, context, scores, semantics, and governance fields.

Validated snapshot: Stage 1 out-of-distribution (OOD) macro-F1 is 0.9478; Stage 2 macro-F1 is 0.7778 on the documented held-out split; replaying 460 flows yields 460 event signals with 42 flows admitted to Stage 2 under the deployed threshold policy. We position this work as an *initial validated input layer*—not a complete automated threat reasoning or response system; episode analytics, trust inference, and AI-assisted triage remain downstream consumers.

關鍵字: TLS/HTTPS, encrypted traffic, security event signal, edge defense, two-stage classification, operational validation.

1. Introduction

Enterprise, cloud, and industrial edges now carry predominantly encrypted sessions. Where bulk decryption is infeasible or policy-disallowed, defenders observe TLS handshakes, record sizes, inter-arrival times, and coarse metadata—signals that are legally and technically accessible but weakly aligned with traditional application-layer antivirus taxonomies.

Gap. A rich literature improves *encrypted traffic classification*, yet many published systems stop at an argmax label. In contrast, SOCs, incident responders, and orchestration fabrics consume *events*: artifacts with stable identifiers, timestamps, observable context, calibrated confidence, tactic-relevant semantics, and lineage to models and datasets. Without such structure, classifiers are hard to replay after model updates, difficult to correlate across hosts, and risky to feed into automated playbooks.

Positioning. We treat the learning stack as *replaceable*: commodity flow export (e.g., IPFIX-style summaries [2]) and learned representations [3] are implementation details behind a fixed *event interface*. Our scientific question is operational: *How should encrypted flow predictions be serialized so training-time validation, online inference, and offline replay agree on dispatch and final fields?*

Approach. Stage 1 performs benign/malicious screening with probability calibration and a single validated threshold policy shared across train, serve, and replay. Stage 2 assigns finer threat-behavior semantics only to flows that satisfy that gate, reflecting asymmetric compute budgets at the edge. Each flow maps to the schema in Section 3..

Snapshot. Stage 1 OOD macro-F1 is 0.9478; Stage 2 macro-F1 is 0.7778; 460 replayed flows produce 460 structured signals with 42 Stage 2 inferences, matching dispatch counters under fixed versions.

Contributions.

- **Two-stage edge screening and behavior labeling** with explicitly shared calibration/threshold semantics between training and production.
- **Structured event-signal schema** aligning inference outputs to downstream correlation and CTI-style fields without vendor lock-in.

- **Replay validation** demonstrating per-flow signal cardinality and Stage 1/Stage 2 admission parity on archived traces.

Temporal feature fusion, open-set discovery, long-term drift adaptation, unseen-protocol guarantees, Edge-SLM copilots, and automated enforcement are *explicitly out of scope* for the present validation (Section 6.).

2. Related Work

We situate the paper along three threads: learning methods for encrypted traffic, sound evaluation under operational bias, and standards-ready evidence exchange; the subsections below review each in turn before contrasting with our event-signal emphasis.

2.1 Encrypted traffic classification

Survey literature classifies encrypted flows using handcrafted statistics, deep models over bytes or bursts, and learned sequence encoders [1]. Pre-training followed by fine-tuning improves sample efficiency and transfer across tasks [3, 9]. These works advance *representation quality*; our focus is instead the *operational envelope*: converting model outputs into auditable security events.

2.2 Evaluation practice and robustness

Encrypted classifiers are sensitive to dataset provenance, split protocols, and environment shifts [8]. Noisy labels and scarce malicious HTTPS traces complicate training [7]; robustness studies highlight performance collapse under dynamic network conditions [6]. Our evaluation therefore reports an OOD test draw with zero cross-split group overlap for Stage 1 and complements classifier metrics with threshold-consistent replay.

2.3 Semantics, telemetry export, and security events

Flow and session records are routinely exported using standard IPFIX-style fields for analytics pipelines [2]. Downstream reasoning benefits when events align to shared threat semantics (MITRE ATT&CK, STIX) for correlation [4, 5]. We bridge these threads: encrypted-flow inference is treated as an *upstream producer* of STIX-amenable records, not as a stand-alone classifier leaderboard entry.

3. System Design

This section states the engineering contract for an edge-native pipeline: what problem we solve, how layers compose from observations to serialized events, and which labels and schema fields downstream tools may rely on.

表 1 Two-stage label contract (validated scope).

Stage / label	Role
Stage 1 benign / malicious	Screening + dispatch gate
Stage 2 C2	Suspicious control channel semantics
Stage 2 data exfiltration	Abnormal outbound movement semantics
Stage 2 HTTPS tunneling	Covert channel over web TLS
Stage 2 domain-fronting-like	Destination-obfuscation semantics

3.1 Goals and Problem Statement

Edge-native defenses must (i) operate without payload visibility, (ii) emit machine-readable evidence richer than a scalar label, and (iii) allow telemetry ingress and model backbones to evolve without breaking downstream automation. We pose:

> *Given an encrypted flow observed at the edge, how can a two-stage inference cascade convert it into a threshold-consistent and semantically meaningful security event signal?*

Non-goals. We do *not* claim full episode reconstruction, global trust calibration, automated containment, or conversational triage; those modules consume—but lie outside—the validated artifact.

3.2 Pipeline Overview

Figure 1 situates four layers. **Layer 0** aggregates heterogeneous observations: programmable telemetry, gateways, SPAN/mirror ports, curated public captures, and controlled HTTPS/TLS threat scenarios. **Layer 1** builds flow records: five-tuple, timing, optional TLS metadata, sequences of lengths/directions/inter-arrival times (for future temporal models), and lineage fields (collection run, dataset id). **Layer 2** runs Stage 1 screening and Stage 2 behavior classification under a shared dispatch policy (Section 4.). **Layer 3** materializes the event tuple with checksums, schema/model versions, and consumer hooks.

3.3 Threat-behavior label space

Table 1 lists the validated taxonomy. Stage 1 isolates malicious candidates; Stage 2 attaches analyst-facing behavior semantics closer to SOC vocabulary than raw application IDs.

3.4 Event signal schema

Equation (1) summarizes the informational bundles embedded in each record; Table 2 enumerates representative keys consumed by downstream SIEM/SOAR

adapters.

$$e_i = \{m_i, n_i, x_i, c_i, a_i, g_i\}. \quad (1)$$

m_i covers identifiers and timestamps; n_i network context; x_i optional feature sequences reserved for temporal extensions; c_i calibrated Stage 1/Stage 2 outputs; a_i tactic/technique hooks; g_i dataset + schema/model hashes.

Designated fields ensure replay can assert bit-identical serialization when inputs and configuration versions match—a requirement for regression testing gateways and analytics rules.

4. Methodology

This section instantiates the design of Section 3.: how features are produced, how Stage 1/Stage 2 models are trained under a single dispatch policy, and how outputs are serialized and replay-checked against that policy.

4.1 Feature materialization

We ingest any source honoring the Layer 1 contract (Section 3.): flows keyed by five-tuple and time window, augmented with TLS metadata when present and optional per-packet summaries for future temporal modules. Feature dictionaries are versioned; hashes over canonicalized tensors are stored in g_i to detect silent skew between training and serving.

4.2 Stage 1 screening and calibration

The Stage 1 classifier outputs malicious probability $\hat{p}$; temperature scaling (or equivalent) maps logits to calibrated $\bar{p}$ on a held-out development split disjoint from OOD evaluation. Class imbalance is countered via reweighting or balanced sampling; the objective is reliable ranking at the operating threshold rather than leaderboard accuracy alone.

4.3 Threshold-consistent dispatch

Let τ denote the malicious admission threshold selected on development replay traces. A flow escalates iff $\bar{p}_{\text{mal}} \geq \tau$. The *same* inequality governs (i) mining supervisory tuples for Stage 2 training, (ii) online inference, and (iii) offline archival replay. This alignment prevents benign label drift (e.g., exporting "malicious" nominal labels while dispatch logic suppresses Stage 2) and is necessary for faithful event emission.

4.4 Stage 2 behavior model

Stage 2 is trained exclusively on flows whose features were generated under the dispatch gate, mirroring conditional distributions at deployment. The classifier head targets the Stage 2 taxonomy in Table 1; macro-F1 is emphasized due to class imbalance.

4.5 Signal synthesis and replay

For each flow we populate e_i (Eq. (1)) including top-k logits, calibrated scores, boolean dispatch flags, schema/model version tuples, and deterministic checksums of serialized inputs. Replay replays stored features through the frozen cascade; Section 5. verifies one signal per ingested flow and identical Stage 2 cardinality relative to admitted malicious traffic.

5. Experimental Validation

5.1 Objectives

We answer three validation questions aligned with the event-signal thesis:

- **RQ1:** Under strict group isolation, does Stage 1 retain strong OOD macro-F1 suitable for dispatch?
- **RQ2:** On held-out malicious-focused data, does Stage 2 retain non-trivial macro-F1 despite class imbalance?
- **RQ3:** Does replay reproduce one event signal per input flow with consistent Stage 1/Stage 2 admission?

Success means downstream consumers can trust serialized fields—not that the pipeline autonomously contains incidents.

5.2 Data and protocol

Evaluations fuse controlled laboratory HTTPS/TLS scenarios with public encrypted-traffic corpora (tunneling, DNS-over-HTTPS families, etc.). Stage 1 estimates benign vs. malicious propagation with train/dev/test sizes $(4160, 450, 460)$, class ratio $3744{:}416$ on training, and *zero* split-group overlap—reducing leakage from correlated captures. Stage 2 trains on dispatched flows with split $(414, 88, 93)$; test support per behavior is $(20, 14, 45, 14)$ for C2, data exfiltration, HTTPS tunneling, and domain-fronting-like classes respectively.

All numbers below are *frozen* artifact exports; we do not claim adaptive drift handling or open-set guarantees.

5.3 Results and interpretation

Table 3 aggregates headline metrics. Stage 1 achieves **0.9478** OOD macro-F1, supporting its role as a conservative yet informative gate: analysts receive calibrated context even when flows stay benign, while suspicious flows accumulate structured evidence.

Stage 2 reaches **0.8495** accuracy and **0.7778** macro-F1 on $n = 93$. Macro-F1 reflects imbalance; accuracy complements readability but should not be interpreted as operational utility in isolation. These metrics validate *behavior tagging conditioned on dispatch*, not end-to-end investigative conclusions.

表 2 Representative security event signal field groups.

Group	Examples (non-exhaustive)
Identity / time	`event_id`, `flow_id`, `session_id`, `event_time`, `run_id`
Network context	`src_ip`, `dst_ip`, ports, `protocol`, logical zones
Sequences (optional)	`packet_len_seq`, `direction_seq`, `iat_seq`
Inference	`stage1_label`, `stage1_score`, `stage2_label`, `stage2_score`, `dispatch_reason`
Threat semantics	`threat_behavior`, `tactic`, `technique`, `stix_mapping`
Governance	`schema_version`, `model_version`, `dataset_source`, content checksum
Downstream hooks	`consumer_hint`, `replay_id`, ticket or playbook placeholders

Replay exercises the full serialization path: **460** ingested flows yield **460** signals; Stage 1 tallies **418** benign vs. **42** malicious calls, and all **42** malicious flows invoke Stage 2—demonstrating parity between scoring, dispatch booleans, and serialized event fields. Discrepancies here would break SOAR automations; their absence supports the engineering thesis of threshold-consistent events.

5.4 Limitations

We do not evaluate long-horizon temporal fusion, earliest-prefix decisions, continual learning under concept drift, or detection of entirely novel threat families. Nor do we study Edge-hosted large language models or automated blocking policies consuming these signals. Those directions require consumers and safeguards beyond the scope of this input-layer validation.

6. Conclusion

This paper argued that encrypted threat analysis at the edge should deliver *structured, threshold-consistent security event signals*, not isolated classifier outputs. We presented a two-stage, vendor-neutral pipeline: calibrated Stage 1 screening, conditional Stage 2 behavior labeling, and deterministic serialization aligned with SOC-oriented schema elements. Validated evidence includes Stage 1 OOD macro-F1 **0.9478**, Stage 2 macro-F1 **0.7778**, and replay coverage showing **460** flows → **460** signals with **42** Stage 2 inferences—confirming dispatch-aligned event generation.

The contribution is intentionally narrow: an **initial input layer** for correlation, ticketing, and future reasoning modules. Realizing trustworthy episode analytics, open-world robustness, temporal fusion, Edge-SLM copilots, or automated enforcement demands additional science and governance; this work supplies the normalized event substrate on which those systems can build without re-deriving per-flow semantics from raw telemetry on every iteration.

参考文献

[1] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè. Encrypted traffic classification using machine learning: A survey. *IEEE Communications Surveys & Tutorials*, 21(2):1996–2016, 2019.

[2] B. Claise, B. Trammell, and P. Aitken. RFC 7011: Specification of the IP flow information export (IPFIX) protocol. Internet Requests for Comments, September 2013.

[3] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu. Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference*, pages 633–642, 2022.

[4] MITRE Corporation. Mitre att&ck framework. `https://attack.mitre.org/`, 2025.

[5] OASIS Open. Stix version 2.1 specification. `https://oasis-open.github.io/cti-documentation/stix/intro`, 2020.

[6] Y. Qing et al. Training robust classifiers for classifying encrypted traffic under dynamic network conditions. In *Proceedings of the ACM Conference on Computer and Communications Security*, 2025.

[7] Y. Qing, Q. Yin, X. Deng, Y. Chen, Z. Liu, K. Sun, K. Xu, J. Zhang, and Q. Li. Low-quality training data only? a robust framework for detecting encrypted malicious network traffic. In *Proceedings of the Network and Distributed System Security Symposium*, 2024.

[8] N. Wickramasinghe et al. Sok: Decoding the enigma of encrypted network traffic classifiers. In *Proceedings of the IEEE Symposium on Security and Privacy*, 2025.

[9] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu. Trafficformer: An efficient pre-trained model for traffic data. In *Proceedings of the IEEE Symposium on Security and Privacy*, 2025.

表 3 Validated experimental snapshot (counts are flows unless noted).

Item	Value
Stage 1 train / dev / test	4160 / 450 / 460
Stage 1 train benign:malicious	3744 : 416
Stage 1 group overlap (train–dev–test)	0
Stage 1 OOD macro-F1 (test)	0.9478
Stage 2 train / dev / test	414 / 88 / 93
Stage 2 test accuracy / macro-F1	0.8495 / 0.7778
Stage 2 test support by class†	20 / 14 / 45 / 14
Replay flows / emitted event signals	460 / 460
Replay Stage 1 benign / malicious	418 / 42
Replay flows entering Stage 2	42

†C2, data exfiltration, HTTPS tunneling, domain-fronting-like.

Dummy PDF file

圖 1 Edge-native pipeline from encrypted observations to structured event signals (artwork may be replaced).